Compulsory Assignment #3

Machine Learning and Deep Learning (CDSCO2041C)

Somnath Mazumdar
sma.digi@cbs.dk
Department of Digitalization, Copenhagen Business School

Deadline: Check Canvas LMS

Instructions

1. Write your group members' student IDs in the notebook.
2. All explanations must be included as markdown/text cells within the notebook.
3. The complete solution must be submitted as **one single *Jupyter* notebook**.

Assignment: Perform image classification

You must use the **CIFAR-100 dataset**, which contains 60,000 colour images of size 32×32 pixels across 100 fine-grained classes. The dataset provides 50,000 training images and 10,000 test images. Use **one shared preprocessing pipeline** across all parts. Download the data from here.

1.1. Data Preparation

Note: The flattened feature vectors produced in this section are used for pixel-based classification (Part 1.3). The original spatial image arrays ($32 \times 32 \times 3$) are retained separately as input to the convolutional autoencoder in Part 1.2.

- Load the CIFAR-100 train and test splits
- Create a validation split from the training set using a 90/10 ratio
- Compute the **per-channel mean and standard deviation** using the training set only
- Normalize the training, validation, and test images using these statistics
- Flatten each normalized image into a feature vector of size $32 \times 32 \times 3 = 3072$

1.2. Autoencoder-Based Feature Learning

Construct a convolutional autoencoder and train it on the normalized training images (using the spatial $32 \times 32 \times 3$ format, **not** the flattened vectors).

- Use an encoder-decoder architecture with at least **two convolutional blocks** in the encoder and a symmetric decoder
- Train the model using **reconstruction loss** (e.g., MSE)
- Use the trained encoder to extract latent feature vectors for the train, validation, and test sets

1.3. Classification Using Pixel and Latent Features

Train a simple classifier (e.g., a linear classifier or k-nearest neighbours) on each of the two feature sets to evaluate their relative utility.

- Train and evaluate one classifier on the **flattened pixel features** from Part 1.1
- Train and evaluate one classifier on the **latent features** extracted by the encoder in Part 1.2
- Use the **same classifier type and hyperparameters** for both to ensure a fair comparison

1.4. CNN Model

Implement a convolutional neural network for CIFAR-100 classification using the normalized images (spatial $32 \times 32 \times 3$ format).

- Use **at least three** `Conv2D` layers organised into two or more convolutional blocks
- Apply `ReLU` activations and max pooling after each block
- Use **at least one** fully connected (dense) layer before the final classification layer
- The output layer must predict all **100 classes** using a softmax activation
- Print the number of trainable parameters in the CNN